课程答辩 · 实训项目

海岚 ERP

服装制造企业资源计划 · 从报价到成本的一条完整链路

Vue 3 + TypeScript / Java 21 + Spring Boot 3.5 / MySQL 8 + Redis

19	8	60
步业务主流程	个岗位角色	项数据自检

答辩人

（填写姓名）

团队

7 人 · 前后端分离

日期

2026 年 10 月

- 一条订单跑通 19 步：报价 → 订单 → 采购 → 生产 → 出货 → 应收 → 成本
- 单据只走动作、库存只认流水、金额只由服务端计算
- 60 项 SQL 自检保证「账能对上」，8 个岗位按钮级权限可现场对比

PROJECT OVERVIEW

项目定位：一条链、九个域、三件事

我们做的不是通用 ERP，而是服装代工厂的「订单到成本」那一段。

上游接客户报价与订单，中间管物料、工单与库存，下游接出货、应收与成本。九个业务域：主数据、销售、采购、生产、库存、出货、成本、人事、财务。

解决三个具体问题

订单与物料脱节
接了单不知道缺多少料 → 净缺料视图按 BOM 算到物料

库存账实不符
有台账没流水 → 全部出入库记流水，结存由流水累加

成本算不清
报价靠拍脑袋 → 标准成本试算并冻结，可被下一轮报价引用

一组实数（可现场验证）

325 后端 Java 文件	**43** REST 控制器	**260** 方法级接口
228 生效权限注解	**66** 数据库表	**60** SQL 自检项

另有 9 个视图、186 项菜单（ERP 132 项：1 目录 + 32 页面 + 99 按钮）、8 个岗位、7 个部门与 7 个岗位定义。

> **一句话：我们只做服装代工这一段，但这一段的状态机、动作幂等与库存账实一致，是按能上线的标准做的。**

CORE • BUSINESS FLOW

业务主流程：一条订单跑完 19 步

01 接单与备料

- 报价单成交 → 转订单
- 订单确认冻结色码/价/期
- 净缺料 = 需求 − 现存 − 在途

产出：订单、缺料清单

02 采购与到货

- 采购单 0→1→2→3
- 下单即计入在途
- 分批收货写入库流水

产出：入库流水、应付

03 生产执行

- 按订单款式建工单（冻结）
- 齐套后领料、按工序报工
- 完工按 SKU 入库

产出：工单、计件、成品

04 交付与结算

- 分批发货 → 签收
- 发货即生成应收
- 收款核销、账龄

产出：出货单、应收

05 核算与闭环

- 标准成本试算 → 确认
- 成本快照可复算
- 生成下一轮报价

产出：结算单、新报价

贯穿全流程的三条纪律

① 状态只走动作端点（没有「直接改状态」的接口）　② 库存只认流水，结存由流水累加　③ 金额只由服务端算，前端不参与结算

客户下单　物料到位　成品入库　客户签收　成本确认

STAGE 01—02

接单与备料：从一张报价单到料进仓库

报价单

0 待确认
1 已接受
3 已转单

销售订单

0 草稿
1 已确认
2 生产中

净缺料

BOM 需求
− 已领 − 现存
− 在途

采购单

0 草稿
1 已下单
2 部分到货/3 已到货

收货入库

写库存流水
结存累加
形成应付

报价只能转一次

转单靠「条件更新」：只有已接受的报价能改成已转单，并发下只有一个请求成功；
重复转单返回 2006，整张订单回滚 —— 防止重复生产、重复记账。

草稿不计在途

采购单确认（或审批通过）那一刻才计入在途；收货后已收数量不再算在途，
所以净缺料 = 需求 − 已领 − 现存 − 在途，不会因为「下了单还没到」而重复采购。

收货即入账

收货写库存流水（只增不改）、结存同步累加、按收货金额挂应付；
若下单时已付款，收货不再重复挂应付 —— 避免同一批料两次计账。

STAGE 03

生产执行：工单冻结，领料只认现存

建工单

冻结色码/ BOM / 工价

领料

扣库存 + 领料流水

报工

工序 × SKU 记合格/不良

完工

按 SKU 入成品库

报工的四道约束（都在服务端校验）

不超过计划量

同一 SKU 同一工序，累计合格数 ≤ 工单冻结的计划量

不超过前道工序

本工序累计合格数 ≤ 前一道工序的累计合格数（不能凭空多出来）

不超工资期

报工日期所属月份工资单已确认 → 拒绝补录，要先作废工资单

外发不计件

委外工序只记成本，不生成员工计件；内部报工才按「合格数 × 工价快照」

两个容易被忽略的边界

· 领料只认现存库存：采购在途的量只参与缺料计算，不能拿来领料，否则账上会出现负数
· 完工入库不超过末道工序累计合格数：没做完的工序不能提前入库，防止「工单说完工、库存却没有」

STAGE 04—05

交付与结算：发货即开票，成本是快照

出货单

按未发量选批次
0 待发货 → 1 已发货 → 2 已签收

应收

金额 = 出货量 × 订单单价
到期日 = 发货日 + 客户账期

收款

登记 + 确认 → 写资金流水
应收余额递减，账龄分档

标准成本

材料 + 人工 + 费用
确认后冻结，不可重算覆盖

新报价

引用已确认成本
形成下一轮报价依据

发齐 ≠ 完成

发齐看累计发货量是否达到订购量；完成看是否所有出货单都签收。两个状态、两个动作。

分批发货

按 SKU + 批次记录，同一订单可以多次发货；未发齐不进入准交率分母，逾期未发齐单独列。

标准成本 ≠ 实际成本

用冻结 BOM × 物料参考价 + 计划合格量 × 冻结工价算；退货不冲回已确认成本。

资金闭环（财务页 8 个标签）

总览 | 账户 | 资金流水 | 月度现金流 | 应收 | 应付 | 账龄汇总 | 收付款

发货是一个事务：校验期间 → 校验状态与未发量 → 扣库存 + 写出库流水 → 改出货状态 → 改订单状态 → 生成应收。
任何一步失败全部回滚，所以不会出现「货发了、应收没生成」。

DESIGN PRINCIPLES

三条贯穿全系统的纪律

01 状态只走动作

做法：不给「直接改状态」的接口，
每个动作的 SQL 自带前置状态

效果：并发下只有一个请求能改成功，
其余按失败回滚

现场可查：ErpOrderService.confirm
（UPDATE … WHERE status=0）

被追问「能直接改库吗」：能改，但
改库等于绕过动作，自检第 28 项会 FAIL

02 动作幂等

做法：请求键 + 参数摘要 + 状态位，
统一落 biz_action_request 表

效果：同键重复提交只入账一次；
同键换参数报 2007；换键正常执行

现场可查：ErpStockService.begin
ErpStockMapper.reserveAction

被追问「键谁生成」：前端每次动作
生成 UUID，网络重试复用同一个

03 账实可对账

做法：结存由流水累加，按「物料 + 批次 +
仓库」四元组双向比对

效果：幽灵出入库跑一次自检就暴露，
不用人工翻账

现场可查：sql/self_check.sql 第 3 项
（库存双向对账）

被追问「怎么发现的」：演示库曾有 5 张
已签收出货单没有出库流水，就是对出来的

为什么先讲这三条

因为业务链路会变（加节点、改规则），但这三条是地基：状态不乱、动作不重、账能对上。后面的技术架构与权限设计都是为它们服务的。

ARCHITECTURE

技术架构：分层清楚，业务独立成模块

前端

Vue 3 + TypeScript + Vite + Element Plus

28 个页面（10 个手写 + 配置驱动）· 26 条路由 · 按钮级权限

后端

Java 21 + Spring Boot 3.5 + MyBatis + Spring Security/JWT

43 个控制器 · 260 个接口 · 228 处权限注解 · 51 个事务方法

数据

MySQL 8（66 张表 / 9 个视图）+ Redis（登录态、AI 限流）

库存流水只增不改 · 结存由流水累加 · 视图统一口径

后端 6 个 Maven 模块

ruoyi-common
公共库：统一响应、异常、工具

ruoyi-framework
安全骨架：JWT、权限 Bean

ruoyi-quartz
定时任务（默认关闭）

myfactoryai-system
用户/角色/菜单/部门 + 单据冻结

myfactoryai-manufacturing
业务域：service / mapper / domain

myfactoryai-admin
启动类 + 控制器 + AI 配置

一次请求怎么走（以订单列表为例）

浏览器 → JWT 过滤 → @PreAuthorize → Controller → Service（校验/事务） → Mapper（注解 SQL） → MySQL

VERIFICATION

怎么证明它对：三层验证

60 项 SQL 自检

库存双向对账、单据链路、权限完整度

32 个单元测试

6 个测试类，含权限护栏反射扫描

8 个账号验收

按岗位逐个登录，检查菜单与 403 边界

自检里最能说明问题的一项

库存双向对账：把 st_inventory 的结存与 st_stock_txn 的流水，按（物料类型 + 物料 + 批次 + 仓库）四元组分别求和比较。

只查一个方向会漏掉「有流水却没有结存行」；双向比对才能发现幽灵出入库。这一项跑不过，说明业务动作有绕过流水的地方。

三条已知不足（主动讲）

没有工作流引擎

审批只有通过/驳回，没有多级会签与条件分支

报工没有复核

报工直接生成计件，缺少班长或质检确认环节

没有 CI

32 个用例与 60 项自检是本地手工跑，未接入流水线

补充：前端 npm run build 含 vue-tsc 类型检查，类型不过不出包；后端 mvn test 通过；演示库的 4 个自检脚本可重复执行，每次改完数据都能跑一遍确认账还对得上。

结论：三层验证构成「改完还能对账」的最小保障；目前缺的是自动化流水线和报工复核环节。

THANK YOU

谢谢，请老师提问

现场可演示：报价转订单 · 分批发货 · 财务页两种角色对比 · 业务问答

演示入口

http://127.0.0.1:5173　账号 admin / sales / finance / boss　密码 Demo@2026!

两条对比

同一个财务页：finance 能登记收付款，boss 只能查看

如果要看代码

ErpOrderService（状态机）· ErpStockMapper（防超卖）· api/catalog.ts（配置驱动）

答辩材料在哪

- docs/答辩手册/00-总讲稿
- docs/答辩手册/01-后端篇
- docs/答辩手册/02-前端篇
- docs/答辩手册/03-问答速查

每份都带口播稿与代码位置